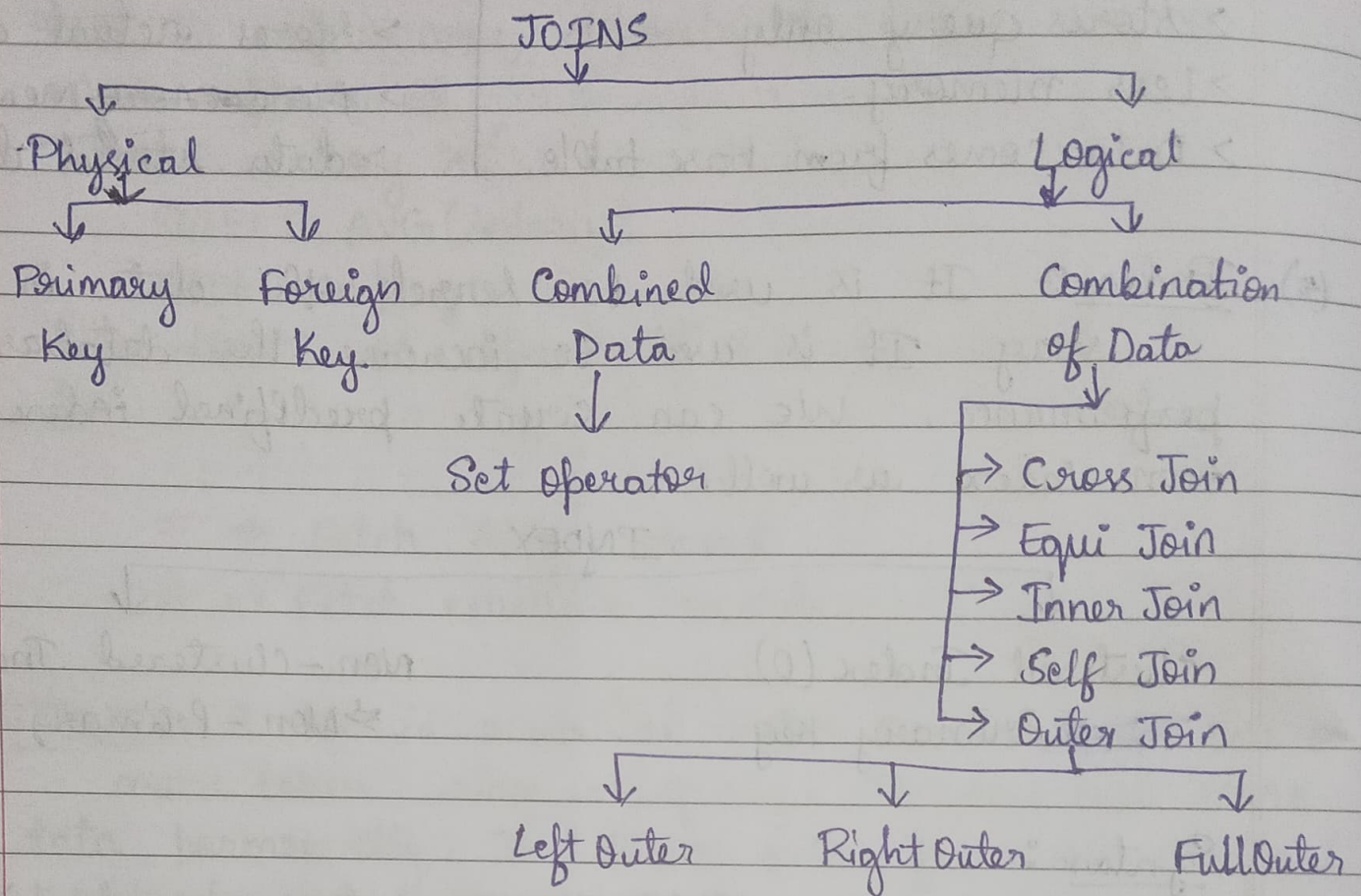


* JOINS:- The process of retrieving data from multiple data table is known as joins.



1. CROSS JOIN:- [Valid and Invalid data].

It is also known as Cartesian joins which is implemented from the set theory of mathematics.

It is also used to fetch the required data from multiple ~~tasks~~ tables.

Combines every row of first table with every row of second table. Returns cartesian product of two tables.

Cross Join can display the output from multiple tables without any connections.

Syntax: SELECT tab. Columns
FROM table 1

CROSS JOIN table 2;
SELECT emp.ename,
dept.dname

FROM emp CROSS JOIN dept;

Note: NO condition is used. Every combination is displayed.

2. EQUI JOIN :- [valid combination].

It removes invalid combination of output.

It has joining condition specified by using '=' relational operator.

It returns matching records based on equal values in common columns.

Syntax: SELECT tab.col-name, tab.col-name
FROM tab-name1, tab-name2
WHERE condition
AND
joining condition.

Example: SELECT e.ename, d.dname
FROM emp e, dept d
WHERE e.deptno = d.deptno;

3. INNER JOIN :- It is same as equi join but it can use all comparative operator including '='. Returns only matching records from both tables.

Syntax: SELECT columns
FROM table1
INNER JOIN table2
ON table1.column-name = table2.column-name;

Example: SELECT emp.emp-id,
emp.emp-name,
dept.dept-name
FROM emp
INNER JOIN dept
ON emp.dept-id = dept.dept-id;

4. SELF JOIN :- A table joined with itself. Used when rows in the same table are related.

Syntax: `SELECT a.column-name,
b.column-name
FROM table-name a
JOIN table-name b
ON a.common-column = b.common-column;`

Example: `SELECT e.emp-name AS Employee,
m.emp-name AS Manager
FROM emp e
JOIN emp m
ON e.manager-id = m.emp-id;`

5. Outer JOIN :-

> LEFT OUTER JOIN :- Returns All rows from the left table, & matching rows from the right table.

If no match, NULLS appear for right table columns.

Syntax: `SELECT columns
FROM table 1
LEFT JOIN table 2
ON table 1.column-name = table 2.column-name;`

> RIGHT JOIN :- Returns ALL rows from the right table, & matching rows from the left table.

If no match, NULLS appear for left table columns.

Syntax: `SELECT columns
FROM table 1
RIGHT JOIN table 2
ON table 1.column-name = table 2.column-name;`

> FULL OUTER JOIN :- Returns all rows from both tables. NULLS appear where there is no match on either

Side. MySQL doesn't support FULL OUTER JOIN directly -
use UNION of LEFT & RIGHT JOIN.

Syntax :
 SELECT columns
 FROM table 1
 LEFT JOIN table 2
 ON table 1.column_name = table 2.column_name
 UNION
 SELECT columns
 FROM table 1
 RIGHT JOIN table 2
 ON table 1.column_name = table 2.column_name;

Example : Employee Table :-

empno	empname	deptno
1	Rio	10
2	Raj	20
3	Ram	10
4	Raja	40

Department Table :-

deptno	dname
10	HR
20	Admin
30	Salesman

Left Join :-

empno	empname	deptno	deptno	dname
1	Rio	10	10	HR
2	Raj	20	20	Admin
3	Ram	10	10	HR
4	Raja	40	null	null

Right Join :-

empno	empname	deptno	deptno	dname
1	Rio	10	10	HR
2	Raj	20	20	Admin
null	null	null	30	Salesman
3	Ram	10	10	HR

Full Outer Join :-

empno	empname	deptno	deptno	dname
1	Rio	10	10	HR
2	Raj	20	20	Admin
3	Ram	10	10	HR
4	Raja	40	null	null
null	null	null	30	Salesman

```
SELECT emp.*, dept.*  
FROM emp  
LEFT JOIN dept  
ON emp.deptno = dept.deptno  
UNION  
SELECT emp.*, dept.*  
FROM emp  
Right JOIN dept  
ON emp.deptno = dept.deptno.
```

Full Join Example

(*) Normalisation:- It is used to organize the data into database. It is used to reduce the data redundancy. It is allowed to eliminate the insertion, updation, deletion anomalies (problem).

(*) Types of Normalization:-

- (1) 1NF - First Normal Form.
- (2) 2NF - Second Normal Form.
- (3) 3NF - Third Normal Form.
- (4) BCNF - Boyce Codd Normal Form.

(1) 1NF (First Normal Form):- If a table want to be in 1NF first it should satisfy the following condition -

- (i) The column name of the table should be unique and related.
- (ii) The domain of the table should be same.
- (iii) There should not be any multi valued attributes.

(2) 2NF (Second Normal Form):- If you want to perform 2NF first it should satisfy -

- (i) All the conditions available in 1NF.
- (ii) There should be functional dependency among the attribute.

Note: Functional dependency is nothing but all the non prime attributes should depend on prime attributes.

(3) 3NF (Third Normal Form):- If a table wanted to be in 3NF it should satisfy -

- (i) All the conditions of 2NF.
- (ii) There should be transitive dependency in the attributes.

Note: An indirect dependency among the column is called transitive dependency.

(4) BCNF (Boyce Codd Normal Form):- It is also known as 3.5NF. It is an advanced version of 3NF. The anomalies / problem to resolve will be resolving in BCNF directly.

(*) Advantages of Normalization:-

- > Reduce Duplicate Data.
- > Improves Data Integrity.
- > Saves Storage.
- > Easier Maintenance.
- > Avoids Anomalies.

(*) Disadvantages of Normalization:-

- > More tables.
- > Complex Joins.
- > Query performance may reduce sometimes.

(*) NORMALISATION:- In SQL/DBMS, Normalization is the process of organizing data in a database to: reduce data redundancy (duplicate data), avoid insertion, update, and deletion anomalies, improve data consistency.

(*) Types of Normal Forms:-

1. 1NF (First Normal Form) $\Rightarrow$ Rules:

- > No multiple values in one column.
- > Each column contains atomic values.
- > No repeating groups.

2. 2NF (Second Normal Form) $\Rightarrow$ Rules:

- > Must be in 1NF.
- > No partial dependency.
- > Non-Key attributes should depend on the whole primary key.

3. 3NF (Third Normal Form) $\Rightarrow$ Rules:

- > Must be in 2NF.
- > No Transitive dependency.
- > Non-Key columns should depend only on the primary key.

(*) 1NF:

Biscuit		
Sid	Sname	Course
1	Raj	C, SQL
2	Rio	Java, C++
3	Ram	C#, SQL

----->

Student			
Sid	Sname	C1	C2
1	Raj	C	SQL
2	Rio	Java	C++
3	Ram	C#	SQL

(*) 2NF:

Student			
Sid	Sname	Pid	Pname
01	Raj	01	Ram
02	Rio	01	Ram
03	Raja	01	Ram

Student		
Sid	Sname	Pid
01	Raj	01
02	Rio	01
03	Raja	01

Professor	
Pid	Pname
01	Ram

(*) 3NF:

Student				
Sid	Sname	Pid	Pname	Sage
01	Raj	01	Ram	18
02	Rio	01	Ram	17
03	Raja	01	Ram	20

Student			
Sid	Sname	Sage	Pid
01	Raj	18	01
02	Rio	17	01
03	Raja	20	01

Professor	
Pid	Pname
01	Ram

(*) BCNF:

Student		
Sid	Sname	Pname
01	Raj	Ram
02	Rio	Ram
03	Raja	Ram

Student		
Sid	Sname	Pid
01	Raj	01
02	Rio	01
03	Raja	02

Professor	
Pid	Pname
01	Ram
02	Ram